Design Patterns

Singleton 3
Factory Method 5
Abstract Factory 8
Builder 13
Prototype 17

Creational patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton

Structural patterns

These patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight

Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.

- Chain of Responsibility
- Command
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

Singleton

creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

✅ Steps to Implement Singleton in Java

1. **Make the constructor `private`** → Prevents direct instantiation.
2. **Create a `static` method** → Provides access to the single instance.
3. **Use a `static` field** → Stores the instance so that the same object is returned every time.

```java
public class Singleton {
    private static Singleton instance; // Static field to store the instance

    private Singleton() {  // Private constructor to prevent instantiation
        System.out.println("Singleton instance created");
    }

    public static Singleton getInstance() {  // Static method to return instance
        if (instance == null) {  // Create instance if not already created
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from Singleton!");
    }
}
```

🚀 Want a Thread-Safe Version?

Use **synchronized:**

```java
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static synchronized Singleton getInstance() { // Thread-safe
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

✅ Qu’est-ce qu’un singleton dans Spring ?

> Dans Spring, un bean singleton signifie qu’une seule instance de la classe est créée pour tout le contexte Spring.
> Tous les composants qui utilisent ce bean partagent la même instance.

✅ Comment Spring le gère ?

Spring crée les beans en singleton par défaut.
Tu n’as rien à faire de spécial. Il suffit d’annoter ta classe avec : @Component // ou @Service, @Repository...

Factory Method

Instead of creating objects manually (new SomeClass()), we use a method that creates the object for us.

- It makes the code flexible (you can change the object type without modifying the main code).
- It helps follow the Open/Closed Principle (open for extension, closed for modification).

```
          +----------------+
          |   Product      |  <-- Interface
          |----------------|
          | + create()     |
          +----------------+
                  ▲
       ┌──────────┴──────────┐
       |                     |
+----------------+   +----------------+
|  ProductA      |   |  ProductB      |
|----------------|   |----------------|
| + create()     |   | + create()     |
+----------------+   +----------------+

         +--------------------+
         |   ProductFactory   |  <-- Factory
         |--------------------|
         | + createProduct()  |
         +--------------------+

              ▲
              |
     +------------------------+
     |    FactoryMethodExample |
     |------------------------|
     |  main()                |
     +------------------------+
```

✅ Définition simple :

> Le **Factory Pattern** est un **pattern de création** qui permet de **déléguer la création d'objets** à une **méthode centrale (la "factory")**, au lieu d'utiliser `new`.

✅ Spring propose 2 "factories" :

Interface	Description
`BeanFactory`	Version minimale, légère. Crée les beans à la demande (lazy).

`ApplicationContext`	Version complète : événements, i18n, resources, scopes, etc.

🎯 En Spring, la "factory", c'est le conteneur de beans :

- `BeanFactory` ou `ApplicationContext`
- Ces interfaces **créent et injectent automatiquement** les objets (beans)

🧠 À retenir pour l'entretien :

> **Spring implémente le Factory Pattern via son conteneur d'inversion de contrôle (IoC).**
> **Tu ne fais plus `new`, c'est le contexte Spring (BeanFactory / ApplicationContext) qui fabrique les objets pour toi, en injectant leurs dépendances.**

1. Define a Common Interface
Create an interface that all products will follow.
This ensures all product types have the same methods.

```java
interface Product {
    void create();
}
```

2. Create Concrete Implementations of Products

- Implement different versions of the product that follow the interface.

```java
class ProductA implements Product {
    public void create() {
        System.out.println("Product A is created");
    }
}

class ProductB implements Product {
    public void create() {
        System.out.println("Product B is created");
    }
}
```

3. Create a Factory Class with a Factory Method

Inside the factory class, add a method that decides which product to create.

```java
class ProductFactory {
    public static Product createProduct(String type) {
        if (type.equalsIgnoreCase("A")) {
            return new ProductA();
        } else if (type.equalsIgnoreCase("B")) {
            return new ProductB();
        }
        throw new IllegalArgumentException("Unknown product type: " + type);
    }
}
```

4. Replace Direct Object Creation with Factory Method

- Instead of writing new ProductA(), use the factory method.

```java
public class FactoryMethodExample {
    public static void main(String[] args) {
        Product product1 = ProductFactory.createProduct("A");
        product1.create();  // Output: Product A is created

        Product product2 = ProductFactory.createProduct("B");
        product2.create();  // Output: Product B is created
    }
}
```

5. (Optional) Use Subclasses Instead of a Switch Statement

- Instead of using if-else in the factory, create subclasses of the factory for each product type.

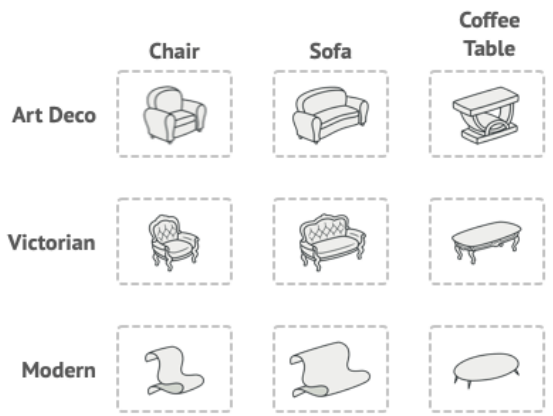
```java
abstract class Creator {
    abstract Product factoryMethod();
}

class ProductACreator extends Creator {
    public Product factoryMethod() {
        return new ProductA();
    }
}

class ProductBCreator extends Creator {
    public Product factoryMethod() {
        return new ProductB();
    }
}
```

Now, we can use:

```java
Creator creator = new ProductACreator();
Product product = creator.factoryMethod();
product.create(); // Output: Product A is created
```

Abstract Factory

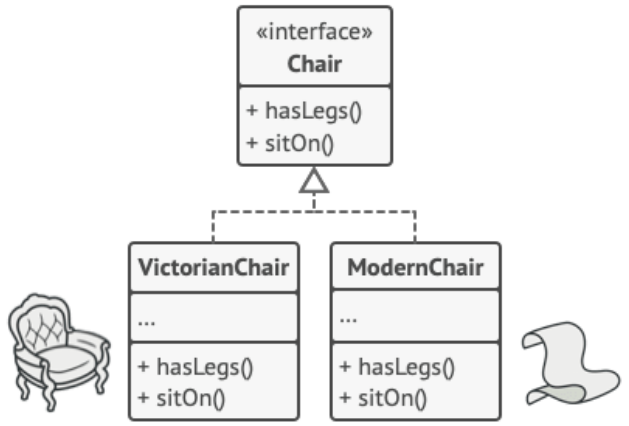
Abstract Factory is a design pattern used to create groups of related objects without needing to know their exact types. It hides the details of how the objects are created.



Product families and their variants.

You need a way to create individual furniture objects so that they match other objects of the same family. Customers get quite mad when they receive non-matching furniture.

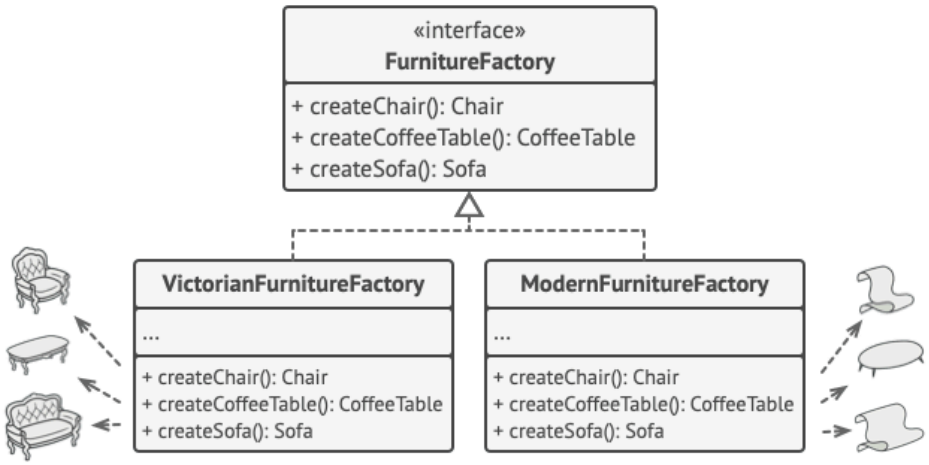
The first thing the Abstract Factory pattern suggests is to explicitly declare interfaces for each distinct product of the product family (e.g., chair, sofa or coffee table). Then you can make all variants of products follow those interfaces. For example, all chair variants can implement the `Chair` interface.



All variants of the same object must be moved to a single class hierarchy.

The next move is to declare the Abstract Factory—an interface with a list of creation methods for all products that are part of the product family (for example, createChair, createSofa and createCoffeeTable). These methods must return abstract product

types represented by the interfaces we extracted previously: Chair, Sofa, CoffeeTable and so on.



Each concrete factory corresponds to a specific product variant.

1. Define product interfaces:

```java
// Chair interface
public interface Chair {
    void sitOn();
}

// Sofa interface
public interface Sofa {
    void lieOn();
}
```

2. **Create concrete products** for each style:

```java
// Modern furniture
public class ModernChair implements Chair {
    public void sitOn() {
        System.out.println("Sitting on a modern chair.");
    }
}

public class ModernSofa implements Sofa {
    public void lieOn() {
        System.out.println("Lying on a modern sofa.");
    }
}

// Victorian furniture
public class VictorianChair implements Chair {
    public void sitOn() {
        System.out.println("Sitting on a Victorian chair.");
    }
}

public class VictorianSofa implements Sofa {
    public void lieOn() {
        System.out.println("Lying on a Victorian sofa.");
    }
}
```

3. **Define the abstract factory**:

```java
public interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
}
```

4. **Create concrete factories** for each furniture style:

```java
public class ModernFurnitureFactory implements FurnitureFactory {
    public Chair createChair() {
        return new ModernChair();
    }

    public Sofa createSofa() {
        return new ModernSofa();
    }
}

public class VictorianFurnitureFactory implements FurnitureFactory {
    public Chair createChair() {
        return new VictorianChair();
    }

    public Sofa createSofa() {
        return new VictorianSofa();
    }
}
```

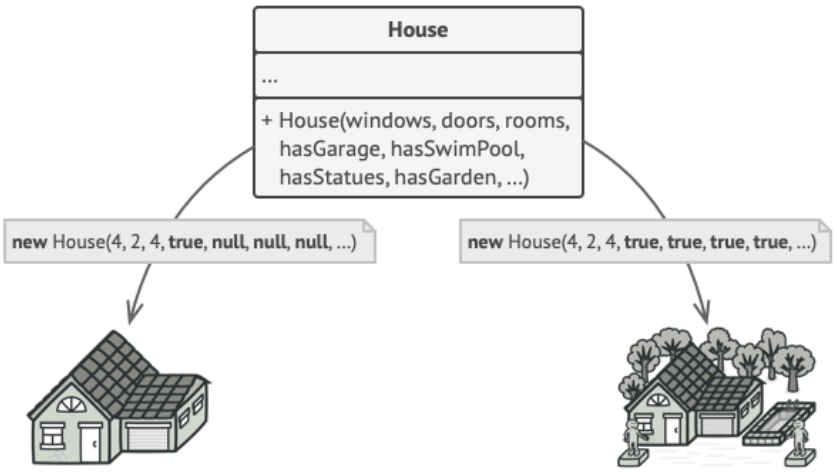
```java
public class FurnitureShop {
    private Chair chair;
    private Sofa sofa;

    public FurnitureShop(FurnitureFactory factory) {
        chair = factory.createChair();
        sofa = factory.createSofa();
    }

    public void showFurniture() {
        chair.sitOn();
        sofa.lieOn();
    }

    public static void main(String[] args) {
        // You can easily switch styles
        FurnitureFactory modernFactory = new ModernFurnitureFactory();
        FurnitureShop modernShop = new FurnitureShop(modernFactory);
        modernShop.showFurniture();

        System.out.println("----");

        FurnitureFactory victorianFactory = new VictorianFurnitureFactory();
        FurnitureShop victorianShop = new FurnitureShop(victorianFactory);
        victorianShop.showFurniture();
    }
}
```

Builder

Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.

For example, let's think about how to create a `House` object. To build a simple house, you need to construct four walls and a floor, install a door, fit a pair of windows, and build a roof. But what if you want a bigger, brighter house, with a backyard and other goodies.

The simplest solution is to extend the base `House` class and create a set of subclasses to cover all combinations of the parameters. But eventually you'll end up with a considerable number of subclasses. Any new parameter, such as the porch style, will require growing this hierarchy even more.



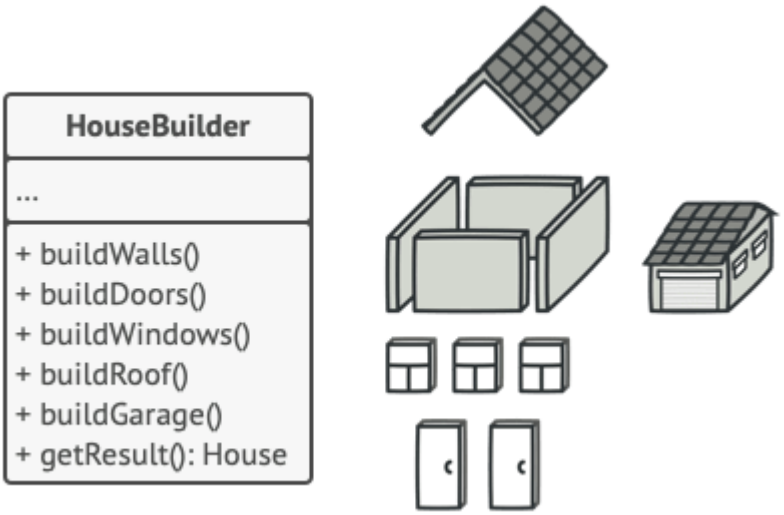
The constructor with lots of parameters has its downside: not all the parameters are needed at all times.

There's another approach that doesn't involve breeding subclasses. You can create a giant constructor right in the base `House` class with all possible parameters that control the house object. While this approach indeed eliminates the need for subclasses, it creates another problem.

In most cases most of the parameters will be unused, making **the constructor calls pretty ugly**. For instance, only a fraction of houses have swimming pools, so the parameters related to swimming pools will be useless nine times out of ten.

Solution

The Builder pattern suggests that you extract the object construction code out of its own class and move it to separate objects called *builders*.



The pattern organizes object construction into a set of steps (`buildWalls`, `buildDoor`, etc.). To create an object, you execute a series of these steps on a builder object. The important part is that you don't need to call all of the steps. You can call only those steps that are necessary for producing a particular configuration of an object.

```java
public class House {
    private String walls;
    private String roof;
    private String door;
    private boolean hasGarden;
    private boolean hasSwimmingPool;

    // Private constructor: only the builder can use it
    private House(Builder builder) {
        this.walls = builder.walls;
        this.roof = builder.roof;
        this.door = builder.door;
        this.hasGarden = builder.hasGarden;
        this.hasSwimmingPool = builder.hasSwimmingPool;
    }

    public void show() {
        System.out.println("House with: " + walls + ", " + roof + ", " + door);
        System.out.println("Garden: " + hasGarden + ", Swimming Pool: " + hasSwimmingPool);
    }
```

```java
// ✅ Static Builder Class inside House
public static class Builder {
    private String walls;
    private String roof;
    private String door;
    private boolean hasGarden;
    private boolean hasSwimmingPool;

    public Builder buildWalls(String walls) {
        this.walls = walls;
        return this;
    }

    public Builder buildRoof(String roof) {
        this.roof = roof;
        return this;
    }

    public Builder buildDoor(String door) {
        this.door = door;
        return this;
    }

    public Builder withGarden() {
        this.hasGarden = true;
        return this;
    }
```

```java
        public Builder withSwimmingPool() {
            this.hasSwimmingPool = true;
            return this;
        }

        public House build() {
            return new House(this);
        }
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        // 🏠 Simple House
        House simpleHouse = new House.Builder()
            .buildWalls("Brick walls")
            .buildRoof("Concrete roof")
            .buildDoor("Wooden door")
            .build();

        simpleHouse.show();

        System.out.println("-----");

        // 🏡 Fancy House with garden and pool
        House luxuryHouse = new House.Builder()
            .buildWalls("Marble walls")
            .buildRoof("Glass roof")
            .buildDoor("Steel door")
            .withGarden()
            .withSwimmingPool()
            .build();

        luxuryHouse.show();
    }
}
```

Prototype

Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

Say you have an object, and you want to create an exact copy of it. How would you do it? First, you have to create a new object of the same class. Then you have to go through all the fields of the original object and copy their values over to the new object.

There's one more problem with the direct approach. Since you have to know the object's class to create a duplicate, your code becomes dependent on that class.

The Prototype pattern delegates the cloning process to the actual objects that are being cloned. The pattern declares a common interface for all objects that support cloning. This interface lets you clone an object without coupling your code to the class of that object.

An object that supports cloning is called a **prototype**. When your objects have dozens of fields and hundreds of possible configurations, cloning them might serve as an alternative to subclassing.

<u>Les 2 grandes façons d'implémenter le **Prototype Pattern:**</u>

1. **Avec `Cloneable` et `clone()` (approche classique)**

```java
static class ProductSheet implements Cloneable {
    private String name;  2 usages
    private String category;  2 usages
    private int price;  3 usages

    public ProductSheet(String category, String name, int price) {  1 usage
        this.category = category;
        this.name = name;
        this.price = price;
    }

    public void setPrice(int price) {  1 usage
        this.price = price;
    }

    public void display() {  2 usages
        System.out.println("Product: " + name + " | " + category + " | " + price + "€");
    }
```

```java
public ProductSheet clone() {
    try {
        return (ProductSheet) super.clone();
    } catch (CloneNotSupportedException e) {
        e.printStackTrace();
        return null;
    }
}
```

```java
    public static void main(String[] args) {
        // Produit original
        ProductSheet original = new ProductSheet( category: "TV",  name: "Electronics",  price: 799);
        ProductSheet clone = original.clone(); // Clonage

        // Modification sur le clone
        if (clone != null) {
            clone.setPrice(699);
        }

        // Affichage
        System.out.println("Original:");
        original.display();

        System.out.println("Clone:");
        clone.display();
    }
}
```

2. Sans `Cloneable`, en créant ta propre méthode de copie

```java
// ✅ Méthode custom au lieu de clone()
public ProductSheet copy() {  1 usage
    return new ProductSheet(this.name, this.category, this.price);
}
```